S2 E13

BK

Season 2: Retrieval and RAG

Field note

Long context or retrieval.

Season 1 showed models read long documents well. Season 2 built a whole pipeline to avoid sending them. Now the whole library fits in the prompt: which approach wins?

Brendan Kelly

Applied AI

Field note

This episode is a field note

No new idea this time. Two lessons already taught, put head to head on the same questions.

LESSON FROM S1 E6 AND E7

Models read long prompts well

Context windows grew huge, and simple lookups worked anywhere in a long document.

LESSON FROM S2 E1

Retrieval sends only what matters

Find the few passages that hold the answer, and send just those.

The open question

Halvard’s whole public library now fits in one prompt. Is the pipeline the season built still worth it, or should you just send everything?

HOW TO READ IT

Question → Setup → Result → Limits → Decision rule

Refresher

Two ways to give the model the library

The same question, the same answer model, the same instruction. Only what goes into the prompt differs.

RETRIEVAL: THE SEASON'S SYSTEM

search the 344 documents

↓

filter, then rerank the top 20

↓

the best 5 passages, about 3,600 tokens

Two model calls in a row: the reranker, then the answer.

LONG CONTEXT: SEND EVERYTHING

no search, no filter, no reranker

↓

all 304 current public documents

↓

the whole library, about 160,000 tokens

One model call, reading everything every time.

Why this is possible now

The answer model reads up to 1,050,000 tokens. Halvard's public library is about 160,000. It fits six times over.

The test: 120 hard and easy questions, both ways

Each question was answered twice: once by the retrieval system, once with the whole library in the prompt.

field-notes/s2-e13-long-context-or-retrieval/longcontext.py — the whole prompt

```
def long_prompt(library_text, item):
    """Everything first, so the provider can cache it,
    then any earlier turns, then the question."""
    parts = [f"Documents:\n\n{library_text}"]
    if item.history:
        parts.append(conversation(item.history))
    parts.append(f"Question: {item.question}")
    return "\n\n".join(parts)
```

Kept the same	The answer model, gpt-6-luna, its instruction and the judge.
A head start	Long context was handed only current, public documents. Retrieval does that sorting itself, with E8’s filter.

Four kinds of question, three of them hard

Three kinds that retrieval found hardest this season, and one easy kind as the control.

Bridge
20 questions

The answer needs two documents: one links the postcode to a region, another gives the region's time.

`"How quickly are urgent safety visits made to my home in postcode area B?"`

Two-part
20 questions

Two questions in one, often answered in two places.

`"How long should I keep my proof of purchase for a Tessa dishwasher from Halvard Home, and how much would it cost to have it collected if I return it?"`

Look-alike
40 questions

Several documents say similar things about different products, or disagree.

`"How long is the warranty on the Brio kettle?"`

Simple
40 questions

One sentence in one policy. The control: both should get these.

`"What extra fee does Halvard Home charge for fitting parts that aren't covered by the warranty?"`

What was expected, written before the run

Four predictions, each with a pass mark set in advance.

More right on hard ones	With everything in front of it, long context beats retrieval on the 80 hard questions.
Same on easy ones	On the 40 simple questions, retrieval loses nothing.
Far pricier	Long context reads at least 20 times as many tokens per question.
Slower	Long context takes at least 3 times as long to answer.

Finding

Both approaches were about as accurate

Right answers, by the judge, on the 120 questions.

	LONG CONTEXT	RETRIEVAL
Bridge of 20	**20**	**14**
Two-part of 20	**17**	**17**
Look-alike of 40	**38**	**40**
Simple of 40	**39**	**39**
All hard ones of 80	**75**	**71**

One by one

9 right only with long context, 5 only with retrieval. A split like that happens by chance about 4 times in 10.

Worked example

Why long context won the bridge questions

A BRIDGE QUESTION

How quickly are urgent safety visits made to my home in postcode area B?

Answer: 32 hours

THE TWO FACTS IT NEEDS

1. "The Midlands service team covers postcode areas B, CV, DE, LE and NG."
2. "Urgent safety visits in Midlands are made within 32 hours."

THE CATCH FOR SEARCH

The question says "B", not "Midlands". Fact 2 matches the question; fact 1 barely does. Search cannot look up the second fact before it knows the first.

Retrieval	The passage linking B to the Midlands ranks 13th, outside the 5 sent. "I could not find how quickly visits are made for postcode area B."	wrong
Long	Both facts are in the prompt. "Urgent safety visits in postcode area B are made within 32 hours."	right

Worked example

Why retrieval won two look-alikes

A LOOK-ALIKE QUESTION

How long is the warranty on the Brio kettle? The warranty policy says 47 months from delivery; a support article says two years from purchase

Retrieval	Both sources reach the top 5. "The passages give conflicting warranty periods: one says two years from the purchase date, while another says 47 months from the delivery date."	right
Long	With 304 documents to choose from, it picks one. "The Brio kettle warranty is two years from the date of purchase."	wrong

The lesson

More context is not more care. Shown only the two relevant passages, the model noticed they disagreed. Shown everything, it answered from one.

Where it breaks

The judge was too generous on two-part answers

Every answer was also checked in code: does it contain the key values, word for word?

	JUDGE	CODE CHECK
Long context two-part, of 20	17	17
Retrieval two-part, of 20	17	11

The 6 disputed	4 answered only one of the two parts, and the judge still marked them right. 2 wrote "£70" where the key says "£70.00".
So	Retrieval’s true two-part score is nearer 13 than 17. Long context’s small lead on hard questions is a little larger than it looks.

Finding

Sending everything was not slower

Median seconds per question, from the first token sent to the last received.

LONG CONTEXT

1.9 s

One call, reading 160,000 tokens.

RETRIEVAL

2.5 s

Two calls in a row: the reranker, then the answer (1.2 s alone).

Why	Modern models read a long prompt fast. Retrieval’s time goes on waiting for one call to finish before the next starts.
With care	The two arms ran in different batches, 8 at once against 4, which may inflate retrieval’s time.

The cost

The real difference: ten times the price

Both arms use the same answer model. Long context pays to read the whole library on every question.

	LONG CONTEXT	RETRIEVAL	RATIO
Tokens read per question	160,398	14,598	11x
Cost per question	$0.016	$0.0016	10x
Cost 100,000 questions a month	$1,606	$157	10x

Where retrieval spends	Most of its 14,598 tokens are the reranker reading 20 passages. The answer call alone reads about 3,650.
Caching	Providers can cache a repeated prompt at a discount. It did not take effect here: 0% cached. If it works, the gap could shrink sharply. Test it.

What sending everything leaves you to solve

Long context only matched retrieval because it was handed a clean library. A real system has to earn that.

Who may see what	Staff pages must never reach a customer's prompt (E8). Send everything, and you must build a separate library per kind of user.
Old versions	Superseded and withdrawn documents must be removed before every question, not just once.
Growth	Cost grows with every document added, on every question. Past about six times this library, it no longer fits at all.

So the pattern is

Retrieval's cost stays roughly flat as the library grows. Long context's grows with it.

Finding

Three of four predictions were wrong

The predictions from slide 6, against the result.

More right on hard ones	75 against 71 of 80: ahead, but within luck.	`failed`
Same on easy ones	39 against 39 of 40.	`held`
Far pricier	11 times the tokens, not 20: retrieval's reranker is a big call too.	`failed`
Slower	1.9 seconds against 2.5: faster, not slower.	`failed`

The lesson

The expectations came from older, slower models with smaller windows. Assumptions about long context age fast: re-test them on each model.

The decision

Which to use, and why retrieval stays

Both work at this size. What decides it is your library, not the accuracy.

LONG CONTEXT FITS WHEN

The documents fit the window with room to spare.

They change rarely.

Every user may see all of them.

Questions often span several documents.

RETRIEVAL FITS WHEN

The library outgrows the window, or is growing.

It changes often.

Access differs by user.

Price per question matters at volume.

The hidden risk

Long context only knows what is pasted in. A user’s own copy of a policy may be out of date, and they act on old information without knowing.

One source of truth

A central system holds the current version for everyone. Update it once, and every answer uses it.

For Halvard

Retrieval stays: the library holds staff-only and superseded documents.

What this does not prove

- **One model, one library size.** Long-context ability varies widely between models.
- **No caching.** A working cache could cut long context’s price, perhaps by most of it.
- **No faithfulness check.** A judge prompt holding the whole library would have cost more than the test.
- **Seconds from different batches,** which may flatter long context’s speed.

For engineers

Run it yourself

The long-context arm, and the comparison with the season's retrieval run.

```
# compare the two arms: no key, no cost
$ uv run python field-notes/s2-e13-long-context-or-retrieval/analyse.py

# the long-context run (keys needed)
$ uv run python field-notes/s2-e13-long-context-or-retrieval/longcontext.py
```

Retrieval arm	Read from E12's reference run: the same system, so no rerun was needed.
Cost	$1.98 for the field note, almost all of it the long-context prompts.

For leaders

Ask whether you need the pipeline at all

Here, sending everything matched a tuned retrieval system on accuracy, at ten times the price per question.

Ask your team

Would our whole knowledge base fit in one prompt today, and at what cost per question?

Ask your team

If we sent everything, how would we keep restricted and outdated documents out?

Ask your team

At our question volume and growth rate, which approach costs less in a year?

Decision rule

Use long context when the documents fit and change rarely; use retrieval when the library outgrows the window, changes often, or must be scoped per user.

Next: S2 E14, Tracing a wrong answer

When the system gets an answer wrong, which stage caused it? Every wrong answer, walked back to where the right passage was lost.

Where this fits

The series map

S1 LLMs	**S2** Retrieval	**S3** Strategy
S4 Adapting	**S5** Agents	**S6** Economics
S7 Data	**S8** Evaluation	**S9** AI teams
S10 Reliability	**S11** Security	**S12** Delivery
S13 Performance	**S14** Resilience	**S15** Governance

☐ Leadership season

Follow linkedin.com/in/brendan-kelly-irl

Episode guide brendankellyai.github.io

Code github.com/BrendanKellyAI/applied-ai-lab